Conquista: un juego de estrategia por turnos como Trabajo Práctico Obligatorio de Programación III

Máximo Bonarrico y Valentín Díaz Imbernón

Facultad de Ingeniería, Universidad Argentina de la Empresa

Programación III (3.4.077), comisión Miércoles Mañana

Docente: Manuel Adrián Cáceres

1 de noviembre de 2026

Resumen

Este trabajo presenta *Conquista*, un sistema desarrollado en Spring Boot y Neo4j que implementa los algoritmos de Programación III como las reglas de un juego de estrategia por turnos. Dos jugadores, una persona y una inteligencia artificial, se disputan un mapa de 12 territorios unidos por 17 rutas cuyo costo son días de marcha. Cada fase del turno es un algoritmo: el cobro usa búsqueda en profundidad y el árbol de recubrimiento mínimo de Prim; la compra, búsqueda binaria y mochila 0/1 con programación dinámica; la marcha, el algoritmo de Dijkstra. La IA rival es un MIN-MAX con poda alfa-beta que evalúa los estados con la matriz de distancias de Floyd-Warshall, calculada una sola vez por partida. Los resultados se verificaron contra cálculos manuales sobre el grafo del grupo y se midió el efecto de cada poda: a profundidad 4, alfa-beta visitó 567 nodos contra 7.408 del MIN-MAX puro (92 % menos) con el mismo valor; el backtracking de rutas exploró 50 nodos con poda contra 93 sin ella; y la mochila compró un 50 % más de poder que el algoritmo voraz con el mismo presupuesto. El sistema se acompaña de una interfaz web jugable y de 89 pruebas automatizadas.

Palabras clave: algoritmos sobre grafos, programación dinámica, backtracking, ramificación y poda, MIN-MAX, poda alfa-beta, juegos de estrategia

Conquista: un juego de estrategia por turnos como Trabajo Práctico Obligatorio de Programación III

El Trabajo Práctico Obligatorio (TPO) de Programación III pide desarrollar un sistema con Spring Boot y Neo4j sobre un dominio propio que permita aplicar con sentido los algoritmos de la materia (Cáceres, 2026). La consigna advierte que el error más costoso es elegir un dominio donde la mitad de los algoritmos quedan forzados: sin un costo numérico en las aristas no hay Dijkstra, Prim ni Kruskal, y sin una selección bajo capacidad no hay mochila ni ramificación y poda.

El grupo comenzó con una red logística de última milla y la reemplazó por un juego de conquista de territorios por turnos, porque un juego le da a cada técnica un papel que el jugador percibe:

- un **costo de arista** natural: los días de marcha que lleva cruzar una ruta;
- un **camino más barato** que importa: llevar tropas al frente antes de que el rival reaccione;
- una **selección bajo capacidad**: qué unidades comprar con el oro del turno;
- un **rival que decide**, que es lo que hace aplicable MIN-MAX. Un juego con azar, como uno con dados, o uno sin turnos no lo permitiría.

Con este planteo, el árbol de recubrimiento mínimo deja de ser un ejercicio aislado: es la red de abastecimiento del imperio y su costo se paga en oro cada turno. El alcance del trabajo cubre los 12 hitos de la cursada, los diez puntos de la rúbrica y los temas adicionales del cronograma: búsqueda binaria, árboles AVL, código de Huffman y MIN-MAX con poda alfa-beta. Incluye una interfaz web jugable y la documentación de cada endpoint con su complejidad justificada.

Marco teórico

Complejidad y recurrencias

La eficiencia de cada algoritmo se expresa con notación asintótica. Las recurrencias por división se resuelven con el teorema maestro: $T(n) = aT(n/b) + O(n^k)$ cuesta $O(n^k \log n)$ cuando $a = b^k$ (Cormen et al., 2022). Así se obtiene $O(n \log n)$ para MergeSort y $O(\log n)$ para la búsqueda binaria.

Divide y vencerás

QuickSort particiona alrededor de un pivote y ordena en O(n log n) en promedio, con un peor caso de $O(n^2)$ (Hoare, 1962). MergeSort garantiza O(n log n) y es estable, a cambio de O(n) de memoria adicional. Los árboles AVL mantienen la altura en O(log n) mediante rotaciones (Adelson-Velsky y Landis, 1962).

Algoritmos voraces

Un algoritmo voraz toma en cada paso la decisión localmente óptima y no la revisa. Es óptimo cuando el problema tiene subestructura óptima y propiedad de elección voraz, como ocurre en Dijkstra (1959), Prim (1957), Kruskal (1956) y el código de Huffman (1952). No lo es en la mochila 0/1, donde el voraz por ratio puede quedar lejos del óptimo (Brassard y Bratley, 1996).

Grafos

Un grafo ralo se representa mejor con listas de adyacencia: los recorridos en anchura (BFS) y en profundidad (DFS) cuestan O(V + E), en lugar del $O(V^2)$ que impone una matriz (Aho et al., 1988).

Programación dinámica

La programación dinámica resuelve problemas con subproblemas superpuestos y principio de optimalidad, guardando cada subproblema en una tabla. La mochila 0/1 cuesta $O(n \cdot W)$ y es seudopolinómica. Floyd-Warshall calcula los caminos mínimos entre todos los pares en $O(V^3)$ mediante una recurrencia sobre el vértice intermedio (Floyd, 1962).

Backtracking y ramificación y poda

El backtracking construye soluciones de forma incremental y retrocede cuando una rama deja de ser viable. La ramificación y poda (*branch and bound*) agrega una cota optimista que descarta las ramas que no pueden mejorar la mejor solución conocida (Horowitz et al., 1996).

Juegos de dos jugadores

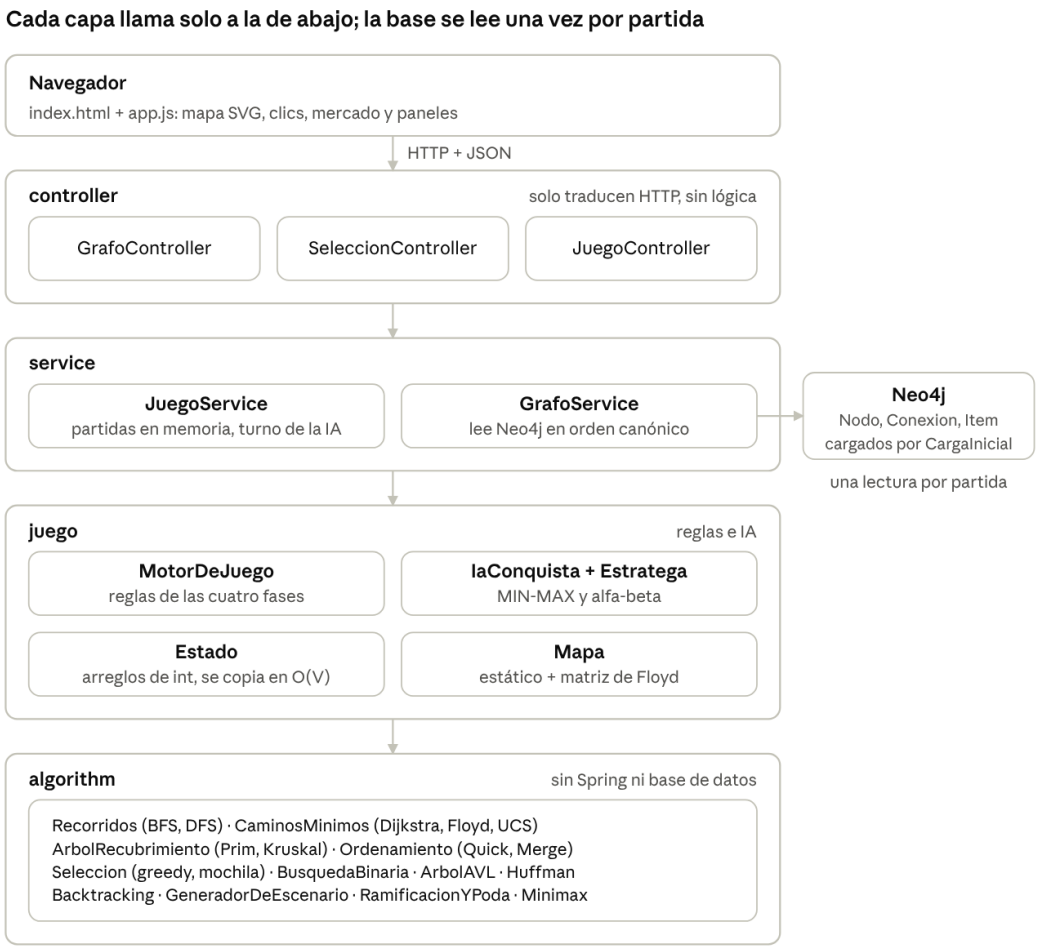
En los juegos de suma cero con información perfecta, MIN-MAX elige la jugada que maximiza el valor suponiendo un rival que lo minimiza, con un costo de $O(b^d)$ para una ramificación b y una profundidad d (Russell y Norvig, 2021). La poda alfa-beta devuelve el mismo valor sin explorar las ramas irrelevantes y, con el mejor orden de jugadas, reduce el costo a $O(b^{d/2})$ (Knuth y Moore, 1975).

Arquitectura del sistema y decisiones de diseño

El sistema separa cinco capas y solo la capa de servicios accede a Neo4j, una vez por partida; todo lo demás se ejecuta en memoria (Figura 1).

Figura 1

Capas del sistema



Nota. Cada capa solo llama a la inmediata inferior. El paquete *algorithm* no depende de Spring ni de la base de datos.

Cinco decisiones sostienen las complejidades declaradas y el funcionamiento de la IA:

- **Lista de adyacencia.** El mapa es ralo (17 rutas para 12 territorios), de modo que BFS y DFS cuestan O(V + E) y no $O(V^2)$.
- **Estado chico y clonable.** El estado dinámico de la partida son unos pocos arreglos de enteros que se copian en O(V). Cada jugada del árbol de MIN-MAX produce una copia, por lo que no se necesita retroceso explícito.
- **Mapa estático compartido.** Los territorios, las rutas, el catálogo ordenado y la matriz de Floyd se calculan una vez por partida; la IA consulta las distancias en O(1).
- **Orden canónico.** Neo4j no garantiza el orden de lectura y de ese orden dependen BFS, DFS y el escenario inicial. El servicio de grafos ordena nodos y rutas por identificador con el MergeSort propio, de modo que la API y las pruebas producen los mismos resultados.
- **Sin lógica en los controladores ni consultas en los algoritmos.** Los controladores solo traducen HTTP y los errores se devuelven como respuestas 400 o 422 con un mensaje, nunca como una traza de pila.

Dominio y reglas del juego

El dominio es un mapa de 12 territorios y 17 rutas no dirigidas, simétrico respecto del centro, en el que AZUL (la persona) y ROJO (la IA) compiten por las capitales. La Tabla 1 traduce los conceptos del scaffold al juego y la Figura 2 muestra el mapa con el escenario clásico.

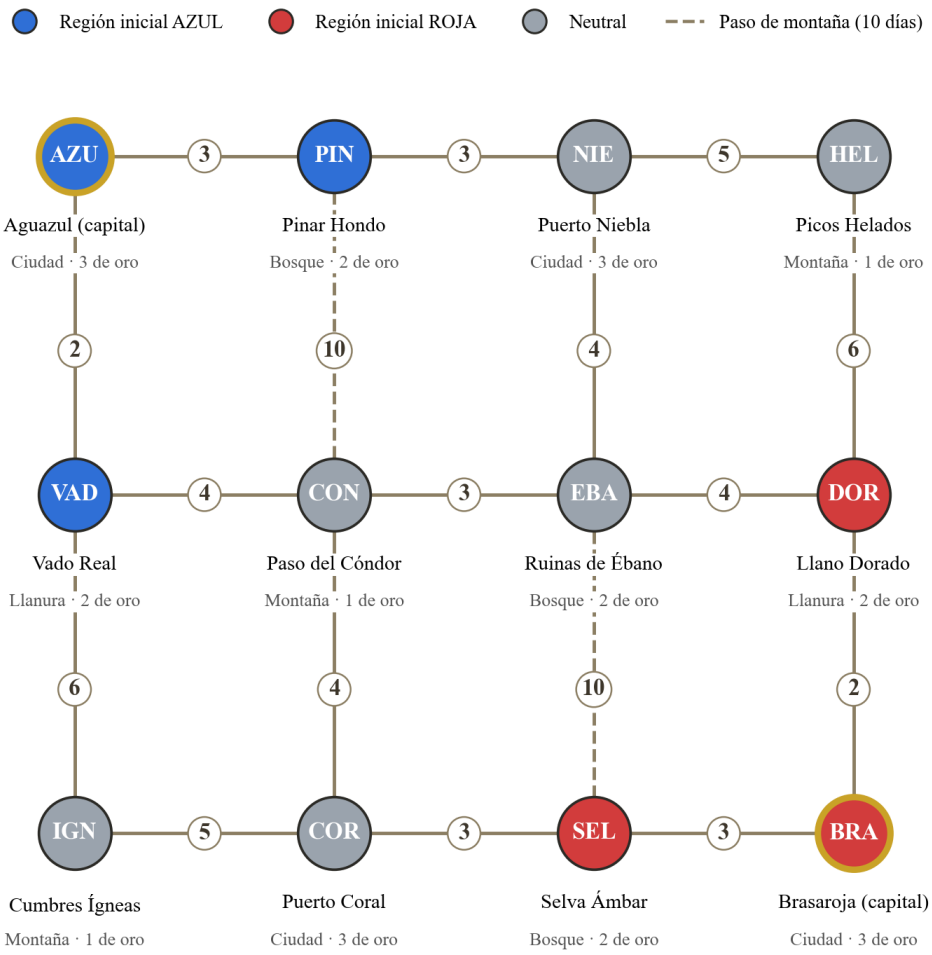
Tabla 1

Correspondencia entre el modelo del scaffold y el dominio del juego

Scaffold	En el juego	Unidad
Nodo	Territorio; *tipo* = terreno, *valor* = producción	oro por turno
Arista (relación CONECTA)	Ruta entre territorios vecinos, recorrible en ambos sentidos	—
Costo de la arista	Tiempo que lleva cruzar la ruta	días de marcha
Item	Unidad del mercado; *peso* = costo, *valor* = poder	oro / tropas

Figura 2

Mapa del juego con el escenario clásico



Nota. Los números sobre las rutas son días de marcha. Cruzar el paso de montaña PIN–CON cuesta 10, pero bordear por el valle PIN–AZU–VAD–CON cuesta 9: es el caso que verifica la relajación de Dijkstra.

Tabla 2

Territorios del mapa

Id	Territorio	Terreno	Producción (oro)	Defensa inicial
AZU	Aguazul (capital azul)	Ciudad	3	9
BRA	Brasaroja (capital roja)	Ciudad	3	9
NIE	Puerto Niebla	Ciudad	3	5
COR	Puerto Coral	Ciudad	3	5
PIN	Pinar Hondo	Bosque	2	4 (azul)
EBA	Ruinas de Ébano	Bosque	2	4
SEL	Selva Ámbar	Bosque	2	4 (roja)
VAD	Vado Real	Llanura	2	3 (azul)
DOR	Llano Dorado	Llanura	2	3 (roja)
CON	Paso del Cóndor	Montaña	1	4
HEL	Picos Helados	Montaña	1	4
IGN	Cumbres Ígneas	Montaña	1	4

Nota. La defensa es tropas + bonus de terreno (llanura 0, bosque 1, ciudad 1, montaña 2) + 2 si es capital.

Tabla 3

Catálogo del mercado

Id	Unidad	Costo (oro)	Poder	Poder por oro
U1	Milicia	1	1	1,00
U2	Arqueros	3	5	1,67
U3	Lanceros	4	6	1,50
U4	Caballería	5	8	1,60
U5	Catapulta	6	9	1,50
U6	Guardia	8	13	1,63

Las cuatro fases del turno

1. **Cobro (automático).** Se suma la producción de los territorios conectados a la capital por territorio propio y se resta el mantenimiento de la red: el costo del árbol de recubrimiento mínimo dividido por 4. Los territorios aislados no producen.
2. **Reclutamiento.** Se compran unidades, una de cada tipo por turno, y se ubican en territorios abastecidos. El sistema sugiere la compra óptima con la mochila 0/1.
3. **Marcha.** Se mueven tropas entre dos territorios propios por el camino más barato a través de territorio propio, siempre que cueste 8 días o menos.

4. **Ataque.** Se ataca un territorio vecino. Si las tropas enviadas superan la defensa, se conquista y quedan *enviadas − defensa*; si no, el atacante las pierde y el defensor pierde la mitad.

El combate es determinista, sin dados, para que MIN-MAX sea aplicable. Gana quien toma la capital rival; si al cabo de 30 rondas no cayó ninguna, gana el jugador de mayor puntaje (10 × producción + tropas). El escenario inicial lo construye un backtracking: capitales en ciudades a 10 días o más, regiones de 3 territorios conexas, sin contacto entre sí y con la misma producción.

Algoritmos implementados

Los temas del cronograma tienen una implementación propia, sin librerías que resuelvan el algoritmo, y cada uno cumple un papel en el juego (Tabla 4). V representa los territorios, E las rutas, n los elementos, W el oro, k los frentes, b las jugadas por estado y d la profundidad.

Tabla 4

Algoritmos, uso en el juego y complejidad

Algoritmo	Uso en el juego	Tiempo	Estructura de datos
DFS	Detectar si el imperio quedó partido (componentes conexas)	O(V + E)	Lista de adyacencia, arreglo de visitados, recursión
BFS por niveles	Territorios alcanzables en k turnos desde la frontera	O(V + E)	Cola FIFO, arreglo de niveles
Dijkstra	Marcha por el camino de menos días por territorio propio	O((V + E) log V)	Cola de prioridad, arreglo de predecesores
Floyd-Warshall	Matriz de distancias precalculada para la IA	$O(V^3)$	Matriz V × V
Prim	Red de abastecimiento desde la capital y su mantenimiento	O(E log V)	Cola de prioridad, arreglo enMst
Kruskal	Verificación de la red con Union-Find	O(E log E)	Heap de aristas, Union-Find
QuickSort	Territorios por valor estratégico	O(n log n) prom.; $O(n^2)$ peor	Partición de Lomuto, mediana de tres
MergeSort	Territorios por amenaza, catálogo y jugadas de la IA (estable)	O(n log n)	Listas auxiliares
Búsqueda binaria	Unidades que se pueden pagar con el oro del turno	O(log n)	Lista ordenada por costo
Árbol AVL	Los k territorios más valiosos dentro de un rango de defensa	O(log n + m log k)	Árbol AVL, heap de mínimos

Algoritmo	Uso en el juego	Tiempo	Estructura de datos
Voraz	Compra por ratio (contraejemplo) y "a quién ataco" por ganancia/riesgo	O(n log n)	MergeSort, acumuladores
Mochila 0/1 (PD)	Compra óptima del turno	O(n·W)	Matriz (n + 1) × (W + 1)
Huffman	Comprimir el registro de la partida	O(n + s log s)	Cola de prioridad de nodos
Backtracking	Rutas de invasión con límite de días y saltos; escenario inicial	O(V!) peor caso	Recursión, visitados, camino parcial
Branch & bound	Reparto de refuerzos entre frentes (máx-mín)	$O(k^n)$ peor caso	Recursión, cargas, cota superior
MIN-MAX	IA rival	$O(b^d)$	Recursión, estado clonable
Alfa-beta	IA rival con poda	$O(b^{d/2})$ con buen orden	Ídem, con (α, β) por valor

Tres decisiones sostienen las complejidades declaradas. La lista de adyacencia mantiene los recorridos en O(V + E) porque cada territorio tiene entre 2 y 4 vecinos. Los heaps binarios sostienen el factor log V de Dijkstra y Prim; con búsqueda lineal del mínimo serían $O(V^2)$. Ningún algoritmo consulta la base de datos: el grafo se carga una vez y se opera en memoria.

La ramificación y poda maximiza el margen del frente más débil, por lo que su cota es superior:

$$\text{cota} = \text{mín}\left(\left(\Sigma_j\, m_j + R\right) / k \;,\; \text{mín}_j\, m_j + R \right)$$

donde m_j es el margen actual de cada frente y R el poder que falta repartir. Ninguno de los dos términos subestima lo que la rama puede dar, de modo que podar cuando la cota no supera a la mejor solución conocida es seguro.

La inteligencia artificial rival

La IA juega con MIN-MAX y poda alfa-beta sobre un árbol en el que cada jugada es un turno completo; a profundidad 4 decide en alrededor de 10 ms. Implementa las tres piezas de un algoritmo de juego:

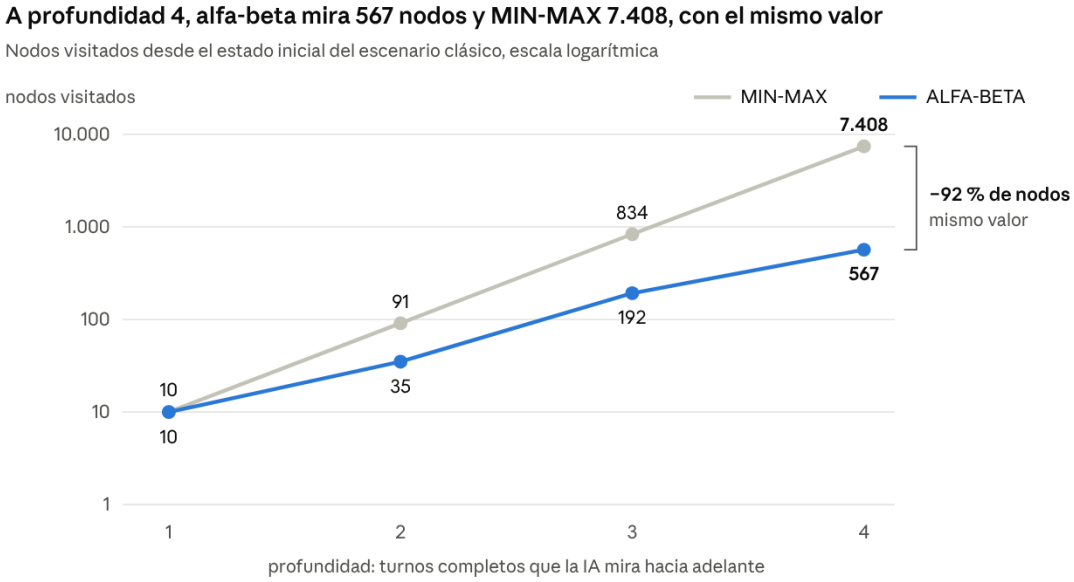
1. **Generador de movimientos.** Cada plan compra lo que indica la mochila y agrega una marcha y un ataque opcionales. Se generan los ataques que ganan (hasta 12), el refuerzo de los dos frentes más amenazados y la fortificación de la capital. Las jugadas llegan ordenadas por la heurística voraz, lo que mejora la poda.
2. **Test de terminal.** Un jugador tomó la capital rival o se llegó a la ronda 30. Una victoria vale 1.000.000 menos la profundidad, para preferir las victorias rápidas.
3. **Función de evaluación.** Es de suma cero, valor(MAX) − valor(MIN), con la expresión siguiente.

$$\text{valor}(p) = 4 \cdot I_p + T_p + 0{,}6 \cdot O_p + 1{,}5 \cdot N_p - 2 \cdot A_p - 0{,}5 \cdot P_p$$

donde I es el ingreso neto, T las tropas, O el oro, N los territorios, A los territorios aislados y P la presión rival sobre la capital, que se calcula con la matriz de Floyd en O(1) por territorio.

Figura 3

Nodos visitados por MIN-MAX y alfa-beta según la profundidad



Nota. Medido sobre el estado inicial del escenario clásico con el endpoint de comparación de la IA. Escala logarítmica.

Las dos curvas de la Figura 3 devuelven el mismo valor y la misma jugada en cada profundidad. MIN-MAX crece casi un orden de magnitud por nivel, $O(b^d)$ con unas 9 jugadas por estado, mientras que alfa-beta crece mucho más lento porque el buen orden de las jugadas acerca su costo a $O(b^{d/2})$.

Tabla 5

Dificultades de la IA

Dificultad	Técnica	Profundidad
Fácil	Voraz por ganancia/riesgo, sin mirar la respuesta del rival	1
Normal	Alfa-beta	2
Difícil	Alfa-beta	4

En simulaciones de IA contra IA sobre el escenario clásico, la dificultad difícil le gana a la normal y a la fácil con ambos colores, y la normal a la fácil.

Verificación y resultados por hito

El contenido de los 12 hitos está implementado y verificado con 89 pruebas automáticas que no necesitan la base de datos; los valores se contrastaron también contra Neo4j en ejecución. La Tabla 6 resume cada hito con su evidencia.

Tabla 6

Hitos, evidencia y valores verificados

Hito	Qué pide la consigna	Evidencia y valor verificado
1	Entorno corriendo y endpoint de resumen	Responde 12 vértices y 17 aristas con el dominio propio
2	QuickSort y MergeSort propios, un test cada uno	Genéricos; 5 pruebas, incluida la estabilidad de MergeSort
3	Grupo, dominio y diagrama de 8 o más nodos con pesos	README y Figura 2: dominio, nodo, arista y costo en días de marcha
4	Dominio cargado en Neo4j, BFS y DFS	BFS(AZU) = AZU, PIN, VAD, CON, NIE, IGN, COR, EBA, HEL, SEL, DOR, BRA
5	Dijkstra con camino, Prim y Kruskal con Union-Find, verificados a mano	Dijkstra(PIN, CON) = 9 por PIN→AZU→VAD→CON y no 10 directo; AZU→BRA = 15; MST de costo 37 con 11 rutas en Prim y Kruskal
6	Mochila 0/1 con tabla y recuperación; caso donde el voraz falla	Con 6 de oro: voraz 6 de poder, mochila 9 (Milicia + Caballería)
7	Floyd con detección de ciclos negativos contra Dijkstra V veces	Floyd: 1.728 combinaciones; Dijkstra × 12: 144 expansiones y 408 relajaciones; mismos 144 resultados
8	Backtracking de rutas con costo y saltos, nodos con y sin poda	AZU→BRA con 22 días o menos y 6 saltos o menos: 7 rutas, 50 nodos con poda contra 93 sin poda
9	Un test por algoritmo y fichas de endpoints	89 pruebas; una ficha por endpoint con ejemplo real y complejidad
10	Informe de complejidades	24 algoritmos con recurrencia, tiempo, espacio y estructura de datos
11	Branch & bound de reparto, cota optimista, nodos podados	Dos columnas: voraz inicial 20, cota 21, óptimo 21; 12 nodos y 4 podas por cota; igual a fuerza bruta en 40 casos
12	Entrega final	Repositorio con el tag entrega-final y el enlace publicado en Teams

Cálculo manual de Dijkstra desde AZU

Tabla 7

Traza de Dijkstra desde AZU

Se cierra	Distancia	Relaja
AZU	0	PIN = 3, VAD = 2
VAD	2	CON = 6, IGN = 8
PIN	3	NIE = 6 (CON por PIN costaría 13: no mejora)
CON	6	COR = 10, EBA = 9
NIE	6	HEL = 11 (EBA por NIE costaría 10: no mejora)
IGN	8	— (COR por IGN costaría 13)
EBA	9	DOR = 13, SEL = 19
COR	10	SEL = 13 (mejora 19)
HEL	11	—
DOR	13	BRA = 15
SEL	13	— (BRA por SEL costaría 16)
BRA	15	—

Nota. El programa reproduce exactamente estas distancias.

Tabla de la mochila con 6 de oro

Tabla 8

Tabla dp de la mochila 0/1 (filas: catálogo ordenado por costo; columnas: oro)

i \ oro	0	1	2	3	4	5	6
Sin unidades	0	0	0	0	0	0	0
U1 (1, 1)	0	1	1	1	1	1	1
U2 (3, 5)	0	1	1	5	6	6	6
U3 (4, 6)	0	1	1	5	6	7	7
U4 (5, 8)	0	1	1	5	6	8	9
U5 (6, 9)	0	1	1	5	6	8	9
U6 (8, 13)	0	1	1	5	6	8	**9**

La recuperación parte de dp[6][6] = 9: U6 y U5 no cambian el valor, U4 sí (9 frente a 7), queda 1 de oro y U1 también. El voraz pierde 3 de poder porque los Arqueros, la mejor unidad por moneda, bloquean la compra que usa todo el presupuesto. Con 10 de oro el voraz logra 14 y la mochila 15; con 14, 19 y 22.

Cumplimiento de la rúbrica

Los diez puntos técnicos de la rúbrica se cumplen al nivel *Completo*: cada algoritmo corre, da resultados correctos sobre los casos de verificación, es una implementación propia, usa una estructura de datos que sostiene la complejidad declarada, tiene pruebas y está documentado (Tabla 9).

Tabla 9

Calificación por algoritmo según la rúbrica

Algoritmo	Puntos	Correcto	Propio	Prueba	Ficha con complejidad
BFS y DFS	2	Sí	Sí	Sí	Sí
Dijkstra, Prim y Kruskal	3	Sí	Sí	Sí	Sí
Voraz	1	Sí	Sí	Sí	Sí
QuickSort y MergeSort	1	Sí	Sí	Sí	Sí
Programación dinámica: mochila 0/1 y Floyd-Warshall	1	Sí	Sí	Sí	Sí
Backtracking	1	Sí	Sí	Sí	Sí
Branch & bound	1	Sí	Sí	Sí	Sí

Tabla 10

Requisitos de admisibilidad y evidencia

Requisito	Evidencia
Hitos, incluidos el 3, 4, 5 y 12	Contenido de los 12 hitos implementado (Tabla 6)
El proyecto compila y las pruebas pasan	89 pruebas sin fallos con ./mvnw test
Ninguna credencial en el repositorio	Las credenciales se leen de variables de entorno; el archivo .env está excluido por .gitignore
README con dominio, nodo, arista y unidad del costo	Sección Dominio del README (días de marcha)
Ficha de cada algoritmo	docs/ENDPOINTS.md, con ejemplo de entrada y salida y complejidad justificada
Participación de los integrantes	Historial de commits de ambos integrantes
Declaración de uso de IA	Sección Uso de IA del README y apartado correspondiente de este documento

Tabla 11

Penalizaciones de la rúbrica

Situación	Estado
Lógica algorítmica en un controlador	No ocurre: los controladores solo traducen HTTP
Algoritmo que consulta la base en su bucle principal	No ocurre: el grafo se carga una vez y se opera en memoria
Endpoint que devuelve una traza de pila	No ocurre: los errores se devuelven como 400 o 422 con un mensaje

Discusión y limitaciones

Modelar el TPO como un juego hizo que cada algoritmo responda una pregunta concreta del jugador, lo que facilita explicarlo y defenderlo. El caso más claro es el árbol de recubrimiento mínimo: expandirse por un paso de montaña agranda el árbol y sube el mantenimiento, por lo que Prim se convierte en una decisión de juego.

La IA tiene tres limitaciones conocidas:

- **Generador de jugadas recortado.** No explora todas las compras, marchas y ataques posibles, sino un conjunto razonable: los ataques que ganan, el refuerzo de los dos frentes más amenazados y la fortificación de la capital, con un máximo de 12 ataques. Sin ese recorte la ramificación crecería sin control.
- **Efecto horizonte.** Con profundidad 4 la evaluación es una estimación. Contra la IA fácil, la difícil a veces intercambia el territorio central durante varios turnos y gana por puntaje en la ronda 30 en lugar de tomar la capital.
- **Planificación por turnos completos.** El análisis y el consejo de la IA solo pueden pedirse al comienzo del turno, antes de comprar o mover.

Por último, la interfaz web se verificó automáticamente contra el servidor en ejecución con un navegador simulado (jsdom), y el mapa es fijo: lo que varía entre partidas es el escenario inicial.

Conclusiones

El sistema implementa y verifica los diez puntos de la rúbrica y cinco temas adicionales sobre un dominio en el que cada técnica tiene un uso concreto. Las mediciones confirman la teoría: alfa-beta obtiene el mismo valor que MIN-MAX con 92 % menos nodos, la poda del backtracking ahorra 46 % de la exploración sin perder rutas y la programación dinámica supera al algoritmo voraz exactamente donde la teoría indica que puede hacerlo.

Como trabajo futuro, la IA podría ganar profundidad con una tabla de transposición, que memoice estados repetidos como la programación dinámica sobre árboles de juego vista en clase, y con un generador de jugadas que contemple ataques coordinados desde varios territorios.

Declaración de uso de inteligencia artificial

Conforme a la sección 7 de la consigna, el grupo declara que utilizó Claude Code (Anthropic, 2026) como asistente para:

- diseñar las reglas del juego de modo que cada fase use un algoritmo de la materia;
- implementar los algoritmos, el motor de juego, la API y la interfaz web;
- redactar las pruebas, la documentación técnica y este documento;
- calcular de forma independiente los valores de verificación y contrastarlos con las pruebas.

La comprensión no se delegó: cada integrante puede explicar el funcionamiento, la complejidad y las estructuras de datos de todos los algoritmos entregados.

Referencias

Adelson-Velsky, G. M., y Landis, E. M. (1962). An algorithm for the organization of information. *Soviet Mathematics Doklady, 3*, 1259–1263.

Aho, A. V., Hopcroft, J. E., y Ullman, J. D. (1988). *Estructuras de datos y algoritmos*. Addison-Wesley.

Anthropic. (2026). *Claude Code* [Software]. https://claude.com/claude-code

Brassard, G., y Bratley, P. (1996). *Fundamentals of algorithmics*. Prentice Hall.

Cáceres, M. A. (2026). *Trabajo Práctico Obligatorio: consignas y rúbrica* [Documento de cátedra]. Programación III, Universidad Argentina de la Empresa.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271. https://doi.org/10.1007/BF01386390

Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM, 5*(6), 345. https://doi.org/10.1145/367766.368168

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal, 5*(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Horowitz, E., Sahni, S., y Rajasekaran, S. (1996). *Computer algorithms*. Computer Science Press.

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE, 40*(9), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898

Knuth, D. E., y Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence, 6*(4), 293–326. https://doi.org/10.1016/0004-3702(75)90019-3

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society, 7*(1), 48–50. https://doi.org/10.1090/S0002-9939-1956-0078686-7

Prim, R. C. (1957). Shortest connection networks and some generalizations. *Bell System Technical Journal, 36*(6), 1389–1401.

https://doi.org/10.1002/j.1538-7305.1957.tb01515.x

Russell, S., y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Anexo A

Endpoints e instrucciones de ejecución

Cada algoritmo es accesible a través de un endpoint. El detalle de cada uno, con un ejemplo real de entrada y salida y la complejidad justificada, se encuentra en el archivo docs/ENDPOINTS.md del repositorio.

Tabla A1

Endpoints por algoritmo

Algoritmo	Endpoint
DFS y componentes	GET /api/grafo/dfs?origen=AZU; GET /api/grafo/componentes
BFS y BFS por niveles	GET /api/grafo/bfs?origen=AZU; GET /api/grafo/alcance?origenes=AZU,PIN,VAD&k=2
Dijkstra y UCS	GET /api/grafo/dijkstra?origen=AZU&destino=BRA; GET /api/grafo/ucs
Floyd-Warshall (hito 7)	GET /api/grafo/floyd; GET /api/grafo/floyd/comparacion
Prim y Kruskal	GET /api/grafo/prim?origen=AZU; GET /api/grafo/kruskal
Backtracking (hito 8)	GET /api/grafo/rutas; GET /api/grafo/rutas/poda; GET /api/juego/escenario
QuickSort y MergeSort	GET /api/seleccion/quicksort?criterio=ratio; GET /api/seleccion/mergesort?criterio=costo
Búsqueda binaria	GET /api/seleccion/binaria?oro=6
Voraz y mochila 0/1	GET /api/seleccion/greedy?capacidad=6; GET /api/seleccion/dinamica?capacidad=6
Branch & bound (hito 11)	GET /api/seleccion/repartir?frentes=2; GET /api/juego/{id}/reparto
Huffman	GET /api/seleccion/huffman?texto=...; GET /api/juego/{id}/replay
Árbol AVL	GET /api/juego/{id}/objetivos?k=3
MIN-MAX y alfa-beta	GET /api/juego/{id}/ia/comparacion?profundidad=4; GET /api/juego/{id}/consejo
Juego	POST /api/juego; POST /api/juego/{id}/comprar, /marchar, /atacar y /terminar

Ejecución

1. Levantar Neo4j, por ejemplo con Docker: docker run -d -p 7474:7474 -p 7687:7687 -e NEO4J_AUTH=neo4j/<contraseña> neo4j:5.
2. Copiar .env.example a .env, completar NEO4J_PASSWORD y exportar las variables. La contraseña nunca se guarda en el repositorio.
3. Ejecutar ./mvnw spring-boot:run. La primera vez se carga el mapa en la base de datos.

4. Abrir http://localhost:8080 para jugar.
5. Ejecutar las pruebas con ./mvnw test; no necesitan Neo4j.